

CS3106L Operating Systems Laboratory

Lab 6: Blocking Mailboxes in LUIT

Graded kernel-concurrency implementation

Department of Computer Science and Engineering, IIT Guwahati

Build a bounded multi-producer/multi-consumer mailbox facility inside the LUIT kernel. The release installs the ABI, boot-time initialization, public programs, submission tooling, and checking infrastructure. The implementation work is confined to `kernel/mailbox.c`.

Central problem:

protect shared state + block without spinning + avoid lost wakeups + survive SMP races

The happy path is only a small part of this lab. Correctness also includes full queues, empty queues, multiple sleepers, close races, queue wraparound, repeated wakeups, and execution on several harts.

1 Required behaviour at a glance

LUIT exposes eight global mailboxes. Every mailbox is a bounded FIFO queue of eight non-negative integer messages.

Operation	Contract
<code>mbox_send(id, value)</code>	Insert <code>value</code> and return 0. Block while the mailbox is full and open. Return -1 for an invalid ID, a negative value, or a mailbox that has been closed.
<code>mbox_recv(id)</code>	Remove and return the oldest buffered value. Block while the mailbox is empty and open. Return -1 for an invalid ID, or after the mailbox is closed and all buffered values have been drained.
<code>mbox_close(id)</code>	Permanently close a valid mailbox and return 0. Repeated close is also 0. Closing never discards buffered values. It must wake operations that can no longer wait for their old condition. Return -1 only for an invalid ID.

The constants are fixed by the supplied header:

```
#define NMAILBOX 8
#define MBOX_CAP 8
```

Valid mailbox IDs are 0 through 7. Valid messages are 0 through 2147483647. The value -1 is reserved as the error/end-of-stream result of `mbox_recv()`.

The API guarantees FIFO order for messages, not FIFO fairness among blocked senders or receivers. LUIT's `wakeup()` may make several sleepers runnable; they compete normally for the mailbox lock and must re-check the predicate. Process-kill/cancellation of a task while it is blocked inside a mailbox syscall is outside the Lab 6 API contract; do not modify LUIT's process-kill machinery for this assignment.

2 What must be true in every execution

For each mailbox, preserve these invariants at all times:

- $0 \leq \text{count} \leq \text{MBOX_CAP}$.
- Queue indices never leave the ring-buffer range.
- A successful receive removes exactly one previously successful send.
- No successful message is duplicated or silently lost.

5. FIFO order is the order in which messages are actually inserted into one mailbox while holding its lock. Two producers that call `mbox_send()` concurrently do not have a predetermined relative order before one of them acquires the mailbox lock.
6. Once `closed` becomes true, it never becomes false until reboot.
7. Closing does not erase already-buffered messages.
8. An operation waiting for a condition must eventually be able to re-check that condition after a relevant state change; it must not miss the wakeup because of an unlock-then-sleep gap.

A lock protects the *invariant*, not merely a line of code. Every field whose consistency participates in the queue state must follow one coherent locking discipline.

3 Release setup

Important: this ZIP is a Lab 6 overlay, not a complete LUIT source tree. It intentionally does not contain LUIT's top-level Makefile. Start from the completed Lab 4 LUIT tree in which make already works; the installer patches that existing Makefile. Do not run the preflight against the extracted Lab 6 package itself.

The course LUIT tree used for this lab does not require `kernel/proc.h` or `kernel/spinlock.h`. Do not create either file and do not copy them from xv6. The supplied `kernel/mailbox.c` includes `kernel/defs.h`; the existing lock implementation remains in `kernel/spinlock.c`.

Use a scratch copy or scratch Git branch of the completed LUIT tree. A Lab 4-complete tree is sufficient; retaining the non-graded Lab 5 practicum additions is optional. First establish the old baseline:

```
make clean
make
make grade
```

Do not install Lab 6 on a tree that is already failing its earlier regression suite.

From the Lab 6 package directory, run the compatibility check against the LUIT tree:

```
python3 tools/lab06_preflight.py /path/to/LUIT_TREE
```

The final line must be:

```
LAB6 preflight: PASS
```

The preflight checks the source-tree assumptions used by this release: the generated syscall ABI, `p->tf`, the existing spinlock discipline, `sleep()/wakeup()`, the process initialization point, `UPROGS`, and the kernel object build rule.

Install the release:

```
python3 tools/install_lab06.py /path/to/LUIT_TREE
```

The installer first verifies every destination and computes all required patches before writing anything. It creates `.lab06_backup/` before patching the tree. It then adds the mailbox ABI, user prototypes, boot initialization, public programs, and release tools, and writes `LAB06_RELEASE.json`. The manifest records both the Lab 6 infrastructure hashes and a source-tree baseline, so later edits outside `kernel/mailbox.c` are detected.

If an installation-time error occurs after writing begins, the installer attempts to restore the original patched files and remove the Lab 6 files it created.

After installation, move to the LUIT root and run:

```
python3 tools/lab06_check.py
make clean
make
```

The starter is intentionally compilable, but mailbox operations return errors until the TODO regions are implemented.

4 Work only in the supplied TODO regions

The implementation file contains five marked regions:

Region	Location	Responsibility
M1	<code>struct mailbox</code>	queue state, closed state, and stable wait-channel state
M2	<code>mailboxinit()</code>	initialize every field for all eight mailboxes
M3	<code>mbox_send()</code>	blocking bounded insertion
M4	<code>mbox_recv()</code>	blocking FIFO removal and closed/drained behaviour
M5	<code>mbox_close()</code>	idempotent close and wakeup of both classes of waiter

Keep every TODO-BEGIN/TODO-END marker. Optional helper functions may be added only between the supplied LAB6-HELPERS-BEGIN/LAB6-HELPERS-END markers. Do not change function signatures, the supplied argument checks, or the syscall wrappers.

After installation, the following files in the LUIT tree are release infrastructure and must remain unchanged. The Makefile listed here is the existing LUIT Makefile patched by the installer; it is not shipped as a standalone file in this ZIP.

```
Makefile
kernel/main.c
kernel/defs.h
kernel/syscall.tbl
kernel/mailbox.h
user/ulib.h
user/mbox_basic.c
user/mbox_block.c
user/mbox_stress.c
user/mbox_multi.c
tools/lab06_check.py
tools/package_lab06.py
tests/grade_lab06.py
docs/lab06_gdb.txt
```

The release checker verifies those files against the install-time manifest. It also checks the source-tree baseline captured at installation time and reports source-like files changed, removed, or added outside the mailbox implementation. Within the mailbox implementation, the supplied scaffold, argument validation, and syscall wrappers are protected; only M1–M5 and the explicit helper region may differ. Generated build products and the course LLM log are excluded from this source-integrity check.

5 Task 1 - Design the per-mailbox state

The starter deliberately gives only the lock and storage array:

```
struct mailbox {
    struct spinlock lock;
    int data[MBOX_CAP];

    /* TODO M1 */
};
```

Add enough state to implement a circular FIFO queue, permanent close, and blocking in two opposite directions.

At minimum, reason about the following logical quantities:

- where the oldest element is located;

- where the next new element will be placed;
- how many elements are currently present;
- whether new sends are still allowed; and
- where receivers and senders sleep.

Use **two distinct, stable, per-mailbox sleep channels**: one for the condition “data may be available” and another for “space may be available”. A sleep channel is an identity (an address); its stored numeric value is not the predicate.

Do not create one global mailbox lock. Each mailbox already has its own lock so activity on mailbox 0 need not serialize activity on mailbox 7.

Checkpoint 1. Before coding M3/M4, be able to state exactly which fields are protected by `mb->lock` and why reading any of those fields without the lock would make a decision based on potentially stale/inconsistent queue state.

6 Task 2 - Initialize the complete subsystem

`mailboxinit()` runs once during kernel boot. Initialize the lock and every state field for every mailbox. Initially:

- the queue is empty;
- the first insertion/removal positions are valid ring-buffer positions; and
- the mailbox is open.

Do not allocate memory dynamically. All mailbox state is static kernel memory.

7 Task 3 - Implement blocking send

`mbox_send(id, value)` already rejects an invalid ID or negative value before the TODO region. Complete the synchronization and queue logic.

A correct design has this shape conceptually, but the missing conditions/actions are yours to implement:

```
acquire(&mb->lock);

while (/* cannot send yet, but waiting could help */)
    sleep(/* writer-side channel */, &mb->lock);

/* Re-check terminal state. */
/* Insert exactly once if sending is still legal. */
/* Wake the class of operation for which data may now exist. */

release(&mb->lock);
```

Important requirements:

1. Never busy-wait on a full mailbox.
2. Never call `yield()` repeatedly as a substitute for blocking.
3. Never release the mailbox lock manually and then call `sleep()`. Use `sleep(chan, &mb->lock)` so becoming asleep is coordinated with releasing the condition lock.
4. Use `while`, not `if`, around a blocking predicate. A wakeup means “state may have changed”, not “this caller owns the newly available slot”.
5. If the mailbox was closed while the sender slept, return -1 without inserting anything.

6. On success, update the circular queue exactly once and wake receivers waiting for data.

A full mailbox contains exactly MBOX_CAP values. Repeated send/receive must wrap indices around the array without walking past it.

8 Task 4 - Implement blocking receive

A receiver has three semantically different cases:

State while holding lock	Required action
queue non-empty	remove and return the oldest value
queue empty, mailbox open	sleep and re-check
queue empty, mailbox closed	return -1

The important ordering rule is **drain before end-of-stream**: a closed mailbox may still contain values. For example:

```
send(17); send(24); send(31); close();
recv() -> 17
recv() -> 24
recv() -> 31
recv() -> -1
```

A successful receive creates one free slot, so it must wake senders waiting for space. Again, a woken sender must re-check the predicate under the lock because another sender may win the slot first.

Checkpoint 2. Explain why this is wrong with two waiting receivers:

```
if (mb->count == 0)
    sleep(...);
/* assume one item now exists */
```

Construct an execution in which both receivers wake for one insertion and the first receiver consumes the only message before the second reacquires the lock.

9 Task 5 - Implement close correctly

Closing is permanent for the lifetime of the boot and is idempotent:

```
close(open mailbox) -> 0
close(closed mailbox) -> 0
```

If a send races with close, use the protected state transition as the ordering point: a send may succeed only if it inserts its value before closed is set while holding the mailbox lock. A sender that observes the closed state after acquiring/reacquiring the lock must return -1 without insertion.

Closing has three simultaneous obligations:

1. prevent every future send;
2. preserve all values that were already buffered; and
3. wake **both** kinds of sleeper.

Why both? An empty closed mailbox gives a sleeping receiver a reason to return -1. A full closed mailbox gives a sleeping sender a reason to stop waiting and return -1. If close wakes only one channel, the other class can remain asleep forever even though its outcome has become final.

Perform the state transition and wakeups while following the same mailbox locking discipline used by send and receive.

10 Task 6 - Preserve the synchronization abstraction

Use the synchronization mechanisms already implemented in LUIT:

```
acquire(...)
release(...)
sleep(chan, lock)
wakeup(chan)
```

Do not modify or bypass them. In particular:

- Do not modify kernel/spinlock.c, kernel/proc.c, the scheduler, context-switch code, or trap code.
- Do not add compiler atomic builtins to kernel/mailbox.c.
- Do not call push_off(), pop_off(), intr_off(), or intr_on() directly from the mailbox implementation. acquire()/release() already implement LUIT's lock/interrupt discipline.
- Do not use arbitrary delays to make a race disappear.
- Do not enlarge MBOX_CAP.
- Do not replace per-mailbox locking with one global lock.

The challenge is to compose the existing primitives correctly, not to rewrite them.

11 Public tests

First run the structural/integrity checker and build:

```
python3 tools/lab06_check.py
make clean
make
```

Then run the public suite. During development, use the quick form:

```
python3 tests/grade_lab06.py --quick
```

Before considering the implementation complete, run the full form:

```
python3 tests/grade_lab06.py
```

The full form runs the existing make grade regression suite before the mailbox tests.

A correct public run ends with:

```
[PASS] mbox_basic: MBOX_BASIC: PASS
[PASS] mbox_block: MBOX_BLOCK: PASS
[PASS] mbox_multi: MBOX_MULTI: PASS
[PASS] mbox_stress: MBOX_STRESS: PASS

LAB6 PUBLIC: PASS
```

What the public programs exercise

- `mbox_basic`: invalid inputs, `INT_MAX`, FIFO order, ring wraparound, idempotent close, send-after-close, and close-then-drain.
- `mbox_block`: a receiver blocked on empty, a sender blocked on full, close waking a blocked receiver, and close waking a blocked sender.
- `mbox_multi`: several simultaneous receivers and senders on `CPUS=4`, one-slot-at-a-time progress that exercises while-based predicate rechecking, and close waking every blocked receiver/sender in the tested groups.
- `mbox_stress`: four concurrent producers and four concurrent consumers on a `CPUS=4` boot; the parent verifies every expected value appears exactly once, with no duplicate or missing message. The full public run repeats this stress workload on fresh boots; `--quick` performs one stress repetition.

Mailbox IDs are intentionally partitioned among the public programs. A closed mailbox remains closed until reboot. The public driver starts fresh QEMU sessions where required and waits for each program to return to the shell after printing its PASS marker. If you run one public program manually a second time, reboot first.

12 Correctness beyond the public tests

Public tests are examples, not the complete contract. The completed implementation must also remain correct under schedules such as:

- several senders sleeping on one full mailbox;
- several receivers sleeping on one empty mailbox;
- a wakeup followed by another caller winning the lock first;
- close racing with send and receive;
- thousands of ring-buffer wraparounds;
- simultaneous operations on different mailboxes;
- repeated `CPUS=4` stress runs; and
- timings that differ from the public programs.

A hang is a correctness failure. A solution that passes once by scheduling luck is not correct.

13 Debugging blocked executions

The package installs `docs/lab06_gdb.txt`. For a focused debugging session:

```
# Terminal A
make qemu-gdb CPUS=2

# Terminal B
gdb-multiarch kernel.elf
```

Useful breakpoints are `mbox_send`, `mbox_recv`, `mbox_close`, `sleep`, and `wakeup`. At `sleep()`, inspect the backtrace: the caller should be the path whose predicate is currently false, and the lock passed to `sleep()` must be the lock that protects that predicate.

Do not call synchronization functions manually from GDB; that changes the state being debugged.

14 Common incorrect designs

Incorrect design	Why it fails
check full/empty, unlock, then sleep	wakeup can occur in the gap; sleeper may never wake
use <code>if</code> around <code>sleep()</code>	multiple/spurious wakeups do not grant ownership of the predicate
spin while full/empty	wastes a hart and violates the blocking contract
keep lock while spinning for opposite operation	can prevent the operation that must change the predicate
one global lock for all mailboxes	serializes independent objects and violates the per-mailbox design
close discards the queue	violates drain-after-close semantics
close wakes only readers or only writers	the other class can remain asleep forever
wake writers after send / readers after receive	wakes the wrong condition and may leave the useful waiter asleep
advance ring index without modulo wrap	eventually indexes outside <code>data[MB0X_CAP]</code>
return from <code>sleep()</code> and proceed without re-check	another process may have changed the state first

15 Create the submission archive

After the full public suite passes, create the submission ZIP from the LUIT root. Replace the example with the exact nine-digit roll number:

```
python3 tools/package_lab06.py 230101037
```

The packaging tool first runs the Lab 6 release checker, verifies that the implementation file is no longer the untouched starter, verifies that all five TODO markers are still present, and then creates exactly one archive:

```
230101037_Lab06.zip
```

The ZIP contains exactly one top-level roll-numbered directory and one source file:

```
230101037_Lab06/
`-- kernel/
   |-- mailbox.c
```

Do not submit the full LUIT tree, object files, disk images, generated syscall files, public tests, screenshots, or additional report files. Submit the verified roll-numbered ZIP through the official course submission channel announced for Lab 6.

16 Final verification checklist

Before the final run, verify all of the following:

- ☐ Only M1–M5 and the explicit helper region in `kernel/mailbox.c` were changed; the supplied wrappers and validation remain untouched.
- ☐ Each mailbox has its own spinlock.
- ☐ The ring-buffer invariants hold at empty, full, and wraparound boundaries.
- ☐ Send blocks rather than spins when full.
- ☐ Receive blocks rather than spins when empty.
- ☐ Blocking predicates are checked with `while`.

- ☐ `sleep()` receives the lock that protects the tested predicate.
- ☐ `Send` wakes receivers; `receive` wakes senders.
- ☐ `Close` wakes both kinds of waiter.
- ☐ Buffered values remain readable after `close`.
- ☐ `New` sends fail after `close`.
- ☐ `Closed-and-drained` `receive` returns `-1`.
- ☐ `python3 tools/lab06_check.py` passes.
- ☐ `make` completes with no warning/error.
- ☐ `make grade` preserves the earlier baseline.
- ☐ The public suite passes on its `CPUS=1` and `CPUS=4` runs, including the multiple-waiter test and repeated full-mode SMP stress runs.
- ☐ The packaging command above creates a verified roll-numbered ZIP containing only `kernel/mailbox.c`.

17 Questions to be able to defend

Be prepared to explain the implementation rather than only its output:

1. Why must a sender re-check count after waking?
2. Why must a receiver re-check count after waking?
3. Exactly what race is prevented by passing `&mb->lock` to `sleep()`?
4. Why are two separate wait channels useful even though one spinlock protects the whole mailbox?
5. Why does closing an empty mailbox wake receivers?
6. Why does closing a full mailbox wake senders?
7. Why is FIFO defined by successful insertion order rather than syscall-entry time when several producers run concurrently?
8. Which invariants would be corrupted if `tail` or `count` were updated outside the lock?
9. Why is a global mailbox lock correct for mutual exclusion but still not an acceptable design here?
10. What is the difference between the job of the mailbox spinlock and the job of `push_off()/pop_off()` inside LUIT's spinlock implementation?

A correct solution should remain correct under a schedule you did not predict.